



Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
MIT Tanszék

Kovács Balázs

FPGA ALAPÚ RENDSZEREK FEJLESZTÉSE HÁZI FELADAT

BUDAPEST, 2026

Tartalomjegyzék

Tartalomjegyzék	2
1. Feladat specifikációja	3
2. Hardver leírás	4
2.1. Modul szintű blokkvázlat.....	4
2.2. Modulok részletes ismertetése	5
2.2.1. RGB2Y modul	5
2.2.2. BRAM sorbuffer-lánc	8
2.2.3. 2D 5x5 FIR szűrő.....	15
2.3. Processzoros alrendszer	25
2.3.1. Blokkvázlat	25
2.3.2. Cím kiosztás.....	26
2.3.3. Implementációs eredmények	26
2.4. Teljes rendszer (top_module)	27
3. Szoftver leírás	33
3.1. A MicroBlaze szoftvere	33
3.1.1. Folyamatábra	34
3.1.2. Megvalósított függvények	35
3.1.3. Kód.....	35
3.1.4. Tesztelés.....	40

1. Feladat specifikációja

A feladat célja egy olyan FPGA-alapú képfeldolgozó rendszer megvalósítása volt, amely valós időben képes egy HDMI bemeneten érkező videójelet feldolgozni, majd a módosított képet HDMI kimeneten továbbítani. A rendszer központi funkciója egy 5×5 méretű FIR szűrő implementálása, amelynek együtthatói futás közben is módosíthatók egy MicroBlaze alapú processzoros alrendszeren keresztül.

A tervezett rendszer működése során a HDMI bemenetből érkező RGB képadatokat először szürkeárnyaltos jellé kell alakítani. Ezt követően a képpontfolyam egy több soros bufferen halad át, amely biztosítja, hogy az aktuális pixel környezetéből előálljon a 5×5 -ös mintavételi ablak. Az így létrejövő pixelkörnyezetet dolgozza fel a FIR szűrő, amely a beállított együtthatók alapján állítja elő a kimeneti pixel értékét.

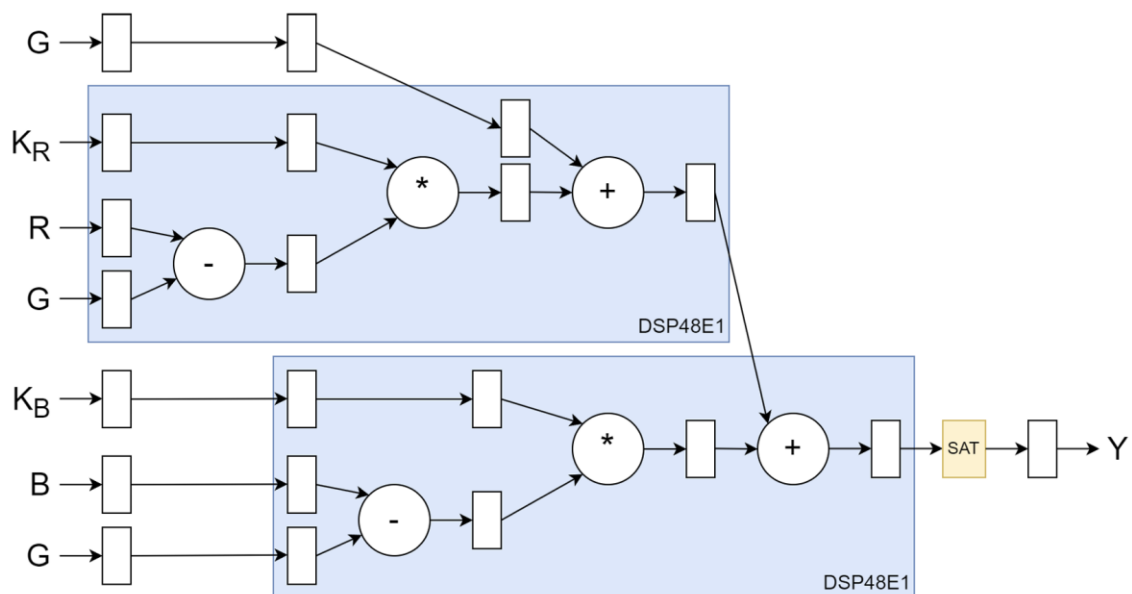
A képfeldolgozás folyamatos adatáramlás mellett kell történnjen, miatt különös figyelmet kellett fordítani a pufferelésre, a jelfeldolgozó pipeline késleltetéseinek kezelésére, valamint a szinkronjelek megfelelő igazítására.

2.2. Modulok részletes ismertetése

2.2.1. RGB2Y modul

A modul feladata a HDMI vevőtől érkező RGB képadatok szürkeárnyalatossá alakítása. A megvalósítás az ITU-R BT.709 szabvány szerinti súlyozást használja, vagyis a vörös, zöld és kék komponensek nem azonos mértékben járulnak hozzá a kimeneti fényességértékhez. A hardveres implementáció során két DSP48E1 blokk került alkalmazásra, mellyel hatékonyan megvalósítható az Y értékre való konverzió. Az együtthatók fixpontos formában szerepelnek, a modul kimenetén pedig szaturáció biztosítja, hogy az eredmény 8 bites pixelértékként biztonságosan továbbadható legyen. Mivel a modul pipeline felépítésű, a képadat és a hozzá tartozó vezérlőjelek késleltetését össze kellett hangolni, ezért a valid, hsync és vsync jelek az adatúttal azonos mértékben késleltetve jelennek meg a kimeneten.

A modul órajelben mért késleltetése: 6 órajel.



2.2.1.1. Modul Verilog kódja

```
module rgb2y(
    input                clk,

    input  signed [17:0] kr_i,
    input  signed [17:0] kb_i,

    input                dv_i,
    input                hs_i,
    input                vs_i,
    input  [7:0]         r_i,
    input  [7:0]         g_i,
    input  [7:0]         b_i,

    output                dv_o,
    output                hs_o,
    output                vs_o,
    output  [7:0]         y_o
);

integer i;
reg [7:0] g_dl[2:0];
// The green component is added back only after the first DSP stage,
// so it must be delayed to match the partial-product pipeline.
always @ (posedge clk)
for (i=0; i<3; i=i+1)
    g_dl[i] <= (i==0) ? g_i : g_dl[i-1];

wire signed [47:0] p_o[1:0];
dsp dsp_0(
    .clk_i    (clk),

    .k_i      (kr_i),

    .pa0_i    (r_i),
    .pa1_i    (g_i),

    // G must be aligned to the same Q17 fixed-point scale as the DSP
    products.
    .pc_i     ({23'b0, g_dl[2], 17'b0}),

    .p_o      (p_o[0])
);

dsp dsp_1(
    .clk_i    (clk),

    .k_i      (kb_i),

    .pa0_i    (b_i),
    .pa1_i    (g_i),

    .pc_i     (p_o[0]),

    .p_o      (p_o[1])
);
```

```

reg [7:0] sat;
always @ (posedge clk)
if (p_o[1][47])
    sat <= 8'b0;
else
    // The fixed-point result is Q17-scaled, so bits [24:17] carry the
    // 8-bit grayscale value once negative results have been clamped.
    sat <= p_o[1][24:17];

assign y_o = sat;

reg [5:0] hs_dl;
reg [5:0] vs_dl;
reg [5:0] dv_dl;
// Keep the control signals aligned with the 6-cycle data path latency
// of the RGB-to-luma conversion, including the final saturation
// register.
always @ (posedge clk)
for (i=0; i<6; i=i+1)
begin
    hs_dl[i] <= (i==0) ? hs_i : hs_dl[i-1];
    vs_dl[i] <= (i==0) ? vs_i : vs_dl[i-1];
    dv_dl[i] <= (i==0) ? dv_i : dv_dl[i-1];
end

assign hs_o = hs_dl[5];
assign vs_o = vs_dl[5];
assign dv_o = dv_dl[5];

endmodule

```

2.2.1.2. Szimulációs eredmények

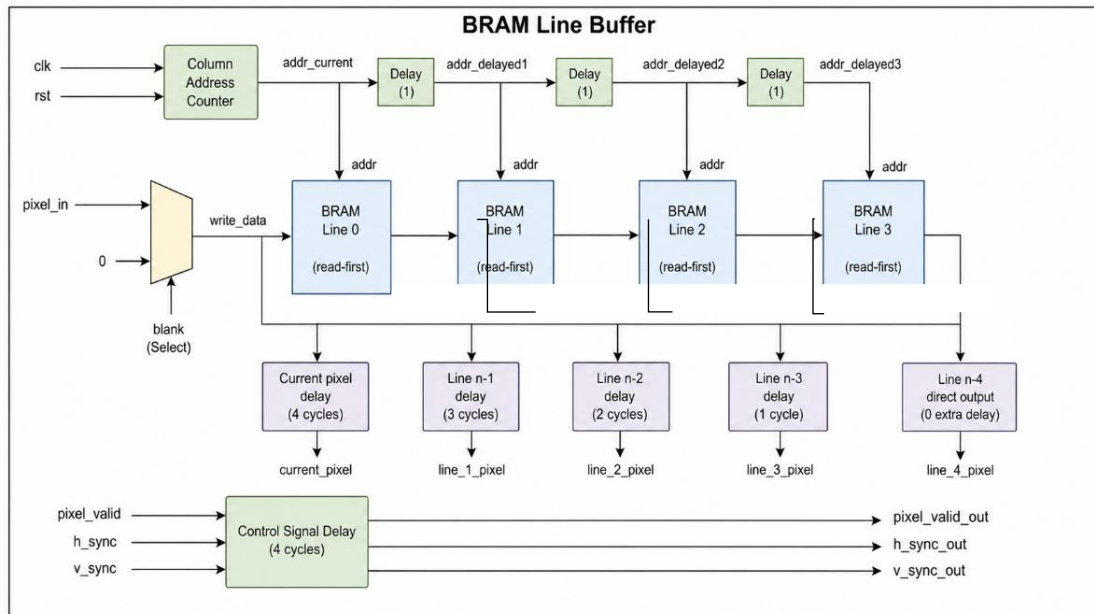
Ennek a modulnak a működését legkönnyebben egy olyan szimulációval lehet ellenőrizni, amelyben a bemenetre egy színes kép pixeleit küldjük és azt az átalakítás után visszaírjuk egy file-ba és ezután összehasonlítjuk azokat. A kimenetnek az eredeti kép fekete-fehér megfelelőjének kell legyen.



2.2.2. BRAM sorbuffer-lánc

A `bram_line_buffer` modul feladata, hogy a beérkező szürkeárnyaltos pixeláramból előállítsa az aktuális pixelhez tartozó, függőleges irányban egymás alatt elhelyezkedő mintákat a 5×5 FIR szűrő számára. A megvalósítás négy egymás után kapcsolt, egy-egy képsort tároló BRAM-alapú ciklikus pufferre épül, amelyek read-first működési módban üzemelnek, így az adott címen korábban eltárolt pixel jelenik meg a kimeneten, mielőtt az új adat ugyanarra a címre beírásra kerülne. A pufferek azonos oszlopindexhez tartozó, megfelelően késleltetett címvezetéknek kapnak, ezért a soronként eltolt adatok helyesen illeszkednek egymáshoz. Érvénytelen képpont esetén a modul nullát ír a memóriába, így a blank szakaszok nem visznek be hasznos képinformációt a pufferláncba. A memóriák induló tartalmát egy initial blokk nullázza, ezért a rendszer kezdeti állapotban is definiált értékekkel működik. A vezérlőjelek késleltetése a pufferlánc adatútjához igazodik, ezért a `pixel_valid`, `h_sync` és `v_sync` jelek a kimeneten megfelelően késleltetve jelennek meg.

A modul órajelben mért késleltetése (a vezérlő jelek alapján) 4 órajel.



2.2.2.1. Modul Verilog kódja

```
/*
 * Single-line storage primitive.
 */
module bram_line_buffer_ram#(

    parameter DATA_WIDTH = 8,
    parameter LINE_WIDTH = 2048,
    parameter ADDR_WIDTH = 11

) (

    input wire clk,
    input wire en,

    input wire [ADDR_WIDTH-1:0] addr,
    input wire [DATA_WIDTH-1:0] din,
    output reg [DATA_WIDTH-1:0] dout
);

reg [DATA_WIDTH-1:0] mem [0:LINE_WIDTH-1];
integer init_idx;

/*
 * Initially set BRAMs to 0 for deterministic behaviour
 */
initial begin
    for (init_idx = 0; init_idx < LINE_WIDTH; init_idx = init_idx + 1)
        mem[init_idx] = {DATA_WIDTH{1'b0}};
    dout = {DATA_WIDTH{1'b0}};
end

always @(posedge clk) begin
    if (en) begin
        // Read First mode
        dout <= mem[addr];
        mem[addr] <= din;
    end
end

endmodule

/*
 * Four cascaded line memories store the previous 4 image rows.
 *
 * The visible outputs are delayed so that current_pixel and the four
 * buffered
 * lines all correspond to the same column when the FIR consumes them.
 */
module bram_line_buffer#(

    parameter DATA_WIDTH = 8,
    parameter LINE_WIDTH = 2048,
    parameter ADDR_WIDTH = 11

) (
```

```

    input  wire          clk,
    input  wire          rst,
    input  wire [DATA_WIDTH-1:0] pixel_in,
    input  wire          pixel_valid,
    input  wire          h_sync,
    input  wire          v_sync,

    output wire [DATA_WIDTH-1:0] current_pixel,
    output wire [DATA_WIDTH-1:0] line_1_pixel,
    output wire [DATA_WIDTH-1:0] line_2_pixel,
    output wire [DATA_WIDTH-1:0] line_3_pixel,
    output wire [DATA_WIDTH-1:0] line_4_pixel,

    output wire          pixel_valid_out,
    output wire          h_sync_out,
    output wire          v_sync_out
);

reg  [ADDR_WIDTH-1:0] column_addr;
wire [ADDR_WIDTH-1:0] addr_current;
wire                write_enable;
wire [DATA_WIDTH-1:0] pixel_or_black;

// The BRAM chain advances every clock; blanked pixels are written as
// zeros.
assign write_enable = 1'b1;
assign pixel_or_black = pixel_valid ? pixel_in : {DATA_WIDTH{1'b0}};

reg  [ADDR_WIDTH-1:0] addr_delayed1;
reg  [ADDR_WIDTH-1:0] addr_delayed2;
reg  [ADDR_WIDTH-1:0] addr_delayed3;

reg                write_enable_delayed1;
reg                write_enable_delayed2;
reg                write_enable_delayed3;

reg                pixel_valid_d1;
reg                pixel_valid_d2;
reg                pixel_valid_d3;
reg                pixel_valid_d4;

reg                h_sync_d1;
reg                h_sync_d2;
reg                h_sync_d3;
reg                h_sync_d4;
reg                h_sync_prev;

reg                v_sync_d1;
reg                v_sync_d2;
reg                v_sync_d3;
reg                v_sync_d4;

// Raw outputs of the four BRAM stages before final tap alignment.
wire [DATA_WIDTH-1:0] line_1_bram_pixel;
wire [DATA_WIDTH-1:0] line_2_bram_pixel;
wire [DATA_WIDTH-1:0] line_3_bram_pixel;
wire [DATA_WIDTH-1:0] line_4_bram_pixel;

// Internal delay chain for aligning the current input pixel to the
// deepest tap.
reg  [DATA_WIDTH-1:0] current_pixel_delayed1;

```

```

reg [DATA_WIDTH-1:0] current_pixel_delayed2;
reg [DATA_WIDTH-1:0] current_pixel_delayed3;
reg [DATA_WIDTH-1:0] current_pixel_delayed4;

// Extra alignment registers for the shallower buffered lines.
reg [DATA_WIDTH-1:0] line_1_pixel_delayed1;
reg [DATA_WIDTH-1:0] line_1_pixel_delayed2;
reg [DATA_WIDTH-1:0] line_1_pixel_delayed3;

reg [DATA_WIDTH-1:0] line_2_pixel_delayed1;
reg [DATA_WIDTH-1:0] line_2_pixel_delayed2;

reg [DATA_WIDTH-1:0] line_3_pixel_delayed1;
wire start_of_line;

// Restart the circular addressing from column 0 on each new line.
assign start_of_line = h_sync & ~h_sync_prev;
assign addr_current = start_of_line ? {ADDR_WIDTH{1'b0}} :
column_addr;

always @(posedge clk) begin
    if (rst) begin
        column_addr      <= {ADDR_WIDTH{1'b0}};

        addr_delayed1    <= {ADDR_WIDTH{1'b0}};
        addr_delayed2    <= {ADDR_WIDTH{1'b0}};
        addr_delayed3    <= {ADDR_WIDTH{1'b0}};

        write_enable_delayed1 <= 1'b0;
        write_enable_delayed2 <= 1'b0;
        write_enable_delayed3 <= 1'b0;

        pixel_valid_d1    <= 1'b0;
        pixel_valid_d2    <= 1'b0;
        pixel_valid_d3    <= 1'b0;
        pixel_valid_d4    <= 1'b0;

        h_sync_d1         <= 1'b0;
        h_sync_d2         <= 1'b0;
        h_sync_d3         <= 1'b0;
        h_sync_d4         <= 1'b0;
        h_sync_prev       <= 1'b0;

        v_sync_d1         <= 1'b0;
        v_sync_d2         <= 1'b0;
        v_sync_d3         <= 1'b0;
        v_sync_d4         <= 1'b0;

        current_pixel_delayed1 <= {DATA_WIDTH{1'b0}};
        current_pixel_delayed2 <= {DATA_WIDTH{1'b0}};
        current_pixel_delayed3 <= {DATA_WIDTH{1'b0}};
        current_pixel_delayed4 <= {DATA_WIDTH{1'b0}};

        line_1_pixel_delayed1 <= {DATA_WIDTH{1'b0}};
        line_1_pixel_delayed2 <= {DATA_WIDTH{1'b0}};
        line_1_pixel_delayed3 <= {DATA_WIDTH{1'b0}};

        line_2_pixel_delayed1 <= {DATA_WIDTH{1'b0}};
        line_2_pixel_delayed2 <= {DATA_WIDTH{1'b0}};

        line_3_pixel_delayed1 <= {DATA_WIDTH{1'b0}};

```

```

end else begin
    h_sync_prev <= h_sync;

    if (start_of_line) begin
        column_addr <= {{(ADDR_WIDTH-1){1'b0}}, 1'b1};
    end else if (write_enable) begin
        if (column_addr == LINE_WIDTH - 1) begin
            column_addr <= {ADDR_WIDTH{1'b0}};
        end else begin
            column_addr <= column_addr + 1'b1;
        end
    end

    // Each later BRAM stage reads the same column one cycle
    later.

    addr_delayed1 <= addr_current;
    addr_delayed2 <= addr_delayed1;
    addr_delayed3 <= addr_delayed2;

    // The delayed enables keep the BRAM pipeline aligned with
    read-first latency.
    write_enable_delayed1 <= write_enable;
    write_enable_delayed2 <= write_enable_delayed1;
    write_enable_delayed3 <= write_enable_delayed2;

    pixel_valid_d1 <= pixel_valid;
    pixel_valid_d2 <= pixel_valid_d1;
    pixel_valid_d3 <= pixel_valid_d2;
    pixel_valid_d4 <= pixel_valid_d3;

    h_sync_d1 <= h_sync;
    h_sync_d2 <= h_sync_d1;
    h_sync_d3 <= h_sync_d2;
    h_sync_d4 <= h_sync_d3;

    v_sync_d1 <= v_sync;
    v_sync_d2 <= v_sync_d1;
    v_sync_d3 <= v_sync_d2;
    v_sync_d4 <= v_sync_d3;

    // Match the current input pixel to the latency of the deepest
    line output.
    current_pixel_delayed1 <= pixel_or_black;
    current_pixel_delayed2 <= current_pixel_delayed1;
    current_pixel_delayed3 <= current_pixel_delayed2;
    current_pixel_delayed4 <= current_pixel_delayed3;

    // The buffered rows need progressively fewer extra delay
    stages.
    line_1_pixel_delayed1 <= line_1_bram_pixel;
    line_1_pixel_delayed2 <= line_1_pixel_delayed1;
    line_1_pixel_delayed3 <= line_1_pixel_delayed2;

    line_2_pixel_delayed1 <= line_2_bram_pixel;
    line_2_pixel_delayed2 <= line_2_pixel_delayed1;

    line_3_pixel_delayed1 <= line_3_bram_pixel;
end
end

```

```

    bram_line_buffer_ram #(
        .DATA_WIDTH(DATA_WIDTH),
        .LINE_WIDTH(LINE_WIDTH),
        .ADDR_WIDTH(ADDR_WIDTH)
    ) bram_line_0 (
        .clk(clk),
        .en(write_enable),
        .addr(addr_current),
        .din(pixel_or_black),
        .dout(line_1_bram_pixel)
    );

    bram_line_buffer_ram #(
        .DATA_WIDTH(DATA_WIDTH),
        .LINE_WIDTH(LINE_WIDTH),
        .ADDR_WIDTH(ADDR_WIDTH)
    ) bram_line_1 (
        .clk(clk),
        .en(write_enable_delayed1),
        .addr(addr_delayed1),
        .din(line_1_bram_pixel),
        .dout(line_2_bram_pixel)
    );

    bram_line_buffer_ram #(
        .DATA_WIDTH(DATA_WIDTH),
        .LINE_WIDTH(LINE_WIDTH),
        .ADDR_WIDTH(ADDR_WIDTH)
    ) bram_line_2 (
        .clk(clk),
        .en(write_enable_delayed2),
        .addr(addr_delayed2),
        .din(line_2_bram_pixel),
        .dout(line_3_bram_pixel)
    );

    bram_line_buffer_ram #(
        .DATA_WIDTH(DATA_WIDTH),
        .LINE_WIDTH(LINE_WIDTH),
        .ADDR_WIDTH(ADDR_WIDTH)
    ) bram_line_3 (
        .clk(clk),
        .en(write_enable_delayed3),
        .addr(addr_delayed3),
        .din(line_3_bram_pixel),
        .dout(line_4_bram_pixel)
    );

    // Output alignment:
    //   current_pixel -> 4 extra cycles
    //   line_1_pixel  -> 3 extra cycles
    //   line_2_pixel  -> 2 extra cycles
    //   line_3_pixel  -> 1 extra cycle
    //   line_4_pixel  -> no extra delay
    assign current_pixel    = current_pixel_delayed4;
    assign line_1_pixel     = line_1_pixel_delayed3;
    assign line_2_pixel     = line_2_pixel_delayed2;
    assign line_3_pixel     = line_3_pixel_delayed1;
    assign line_4_pixel     = line_4_bram_pixel;

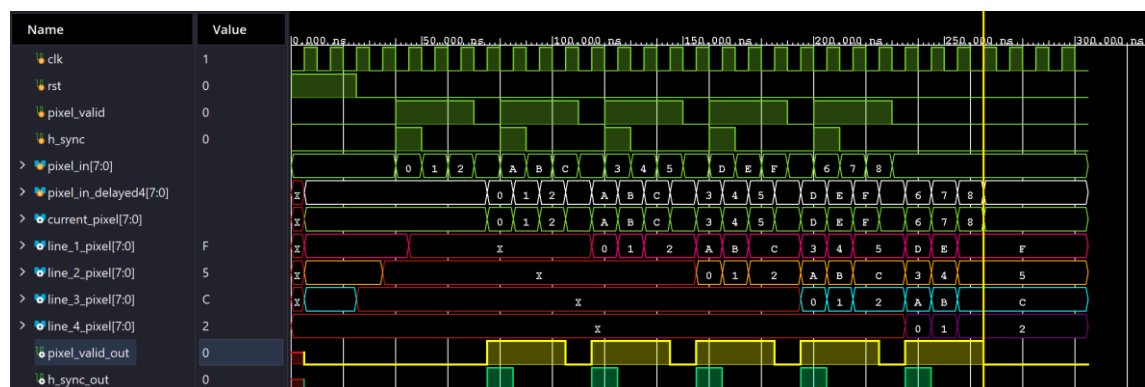
```

```
// Control signals are delayed by the same 4 cycles as the visible
data path.
assign pixel_valid_out  = pixel_valid_d4;
assign h_sync_out      = h_sync_d4;
assign v_sync_out      = v_sync_d4;

endmodule
```

2.2.2.2. Szimulációs eredmények

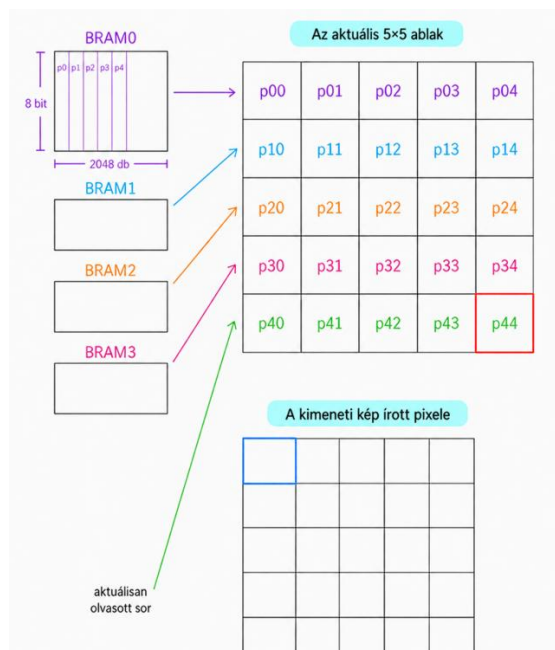
A szimulációhoz kihasználtam, hogy paraméteresen építettem fel a láncban szerepelő BRAM-okat, így kisebb, szimulációban szemmel jobban követhető eseteket is tudtam vizsgálni. A BRAM-ok szín kódolva vannak, miszerint az aktuális sor zöld, az azt megelőző rózsaszín, az azt megelőző narancs, az azt megelőző kék és az azt megelőző lila színnel van megjelölve. Ebben a szimulációban a v-sync nem jelenik meg, mert a buffer feltöltésének és adatmegjelenítésében ez nem játszik szerepet.



Ami látható, hogy 4 órajellel később jelenek meg az adatok a BRAM sorok kimenetén, ehhez megfelelően hozzá vannak igazítva a valid és h-sync jelek és a sorok is egymáshoz vannak igazítva.

2.2.3. 2D 5x5 FIR szűrő

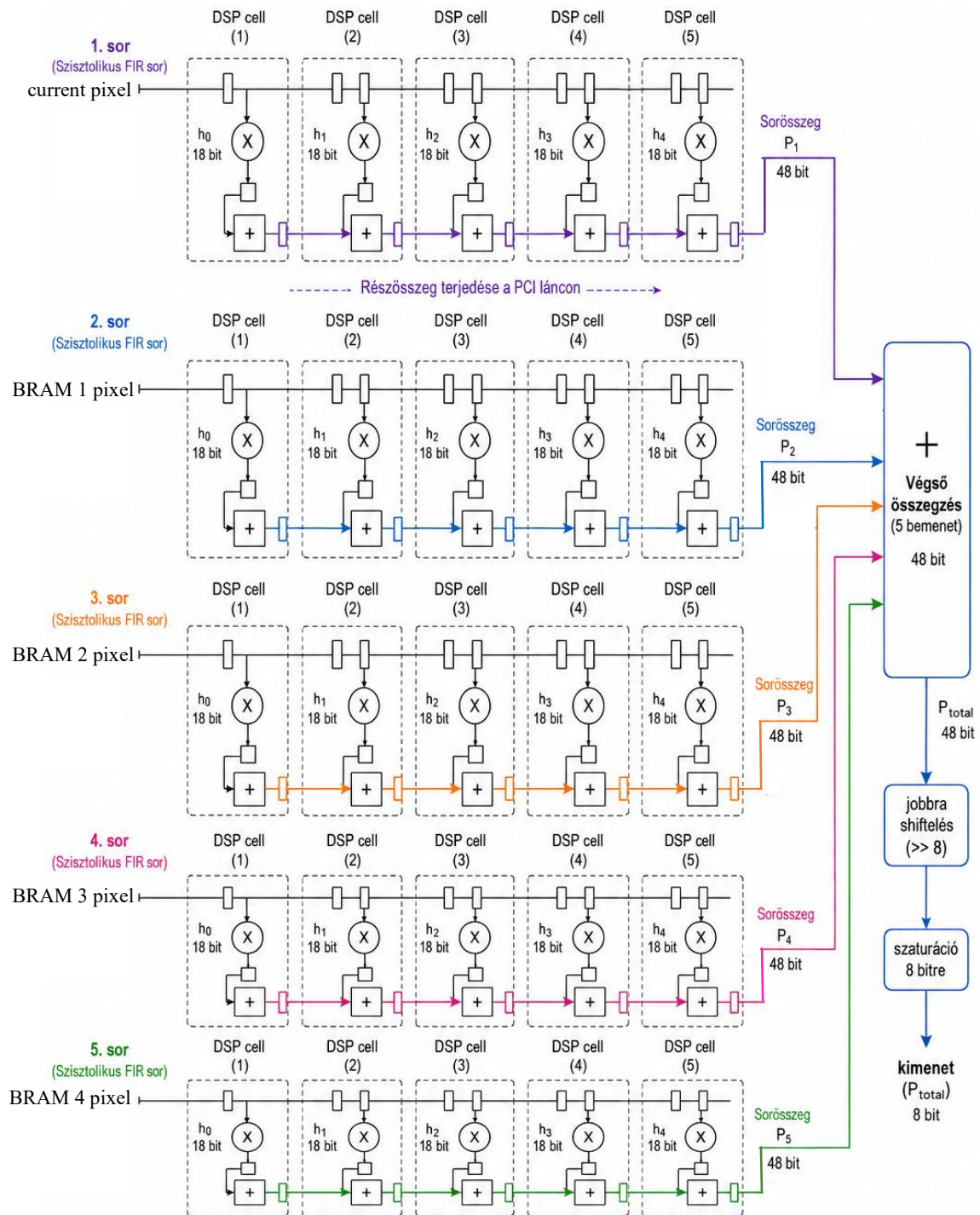
A FIR_filter_5x5 modul feladata, hogy a sorpufferből érkező 5×5 pixeles ablakon elvégezze a kétdimenziós FIR szűrést. A megvalósítás szisztolikus elrendezésű, vagyis a pixelek soronként, csővezetékesen haladnak végig az egyes szorzó-összegző egységeken, ami jól illeszkedik a folyamatos streaming feldolgozáshoz. Az aktuális mintapontnak a 5×5 ablak jobb alsó sarkát választottam, mert ez kauzális feldolgozást tesz lehetővé: a kimeneti érték előállításához így csak az aktuálisan rendelkezésre álló és korábban már beolvasott pixelekre van szükség. A szűrés során először mind az öt sorban külön-külön részösszeg képződik, majd ezek a sorösszegek a modul végén összeadásra kerülnek. Matematikailag a kimenet a következő alakban írható fel:



$$Y = \sum_{i=0}^4 \sum_{j=0}^4 p_{ij} c_{ij}$$

ahol p_{ij} az aktuális 5×5 ablak pixeleit, c_{ij} pedig a hozzájuk tartozó szűrőegyütthatókat jelöli. Mivel a rendszer a HDMI vevő által szolgáltatott rx_clk pixelórán működik, aktív képtartományban minden órajelciklusban egy új pixelt kell feldolgozni, ezért a teljesen streaming architektúrához a 25 szorzás/pixel igény miatt legalább 25 DSP blokk szükséges. A modul végén a felhalmozott eredmény jobbra shifteléssel kerül visszaskálázásra, így a tört részek elhagyhatók, majd szaturáció biztosítja, hogy a HDMI továbbítás felé csak 0...255 közé eső, 8 bites egész pixelérték kerüljön.

A modul órajelben mért késleltetése: 12 órajel.



2.2.3.1. Modul Verilog kódja

```

/*
 * DSP with passthrough for systolic mode
 */
module dsp_25x18_passthrough
# (
    parameter integer A_REG = 2,
    parameter integer B_REG = 2

```



```

) (
    input  wire          clk,
    input  wire signed [24:0] a,
    input  wire signed [17:0] b,
    input  wire signed [47:0] pci,
    output wire signed [24:0] a_passthrough,
    output wire signed [47:0] p
);

reg signed [24:0] a_reg [0:A_REG-1];
reg signed [17:0] b_reg [0:B_REG-1];
reg signed [42:0] m_reg;
reg signed [47:0] p_reg;
integer i;
integer init_idx;

/*
 * For deterministic behaviour set registers to 0 initially
 */
initial begin
    for (init_idx = 0; init_idx < A_REG; init_idx = init_idx + 1)
        a_reg[init_idx] = 25'sd0;
    for (init_idx = 0; init_idx < B_REG; init_idx = init_idx + 1)
        b_reg[init_idx] = 18'sd0;
    m_reg = 43'sd0;
    p_reg = 48'sd0;
end

/*
 * Parameterized registering, this is for easier object marking
 * for the systolic FIR
 * A is for the data
 * B is for the coeff
 */
always @(posedge clk)
for (i = 0; i < A_REG; i = i + 1)
    a_reg[i] <= (i == 0) ? a : a_reg[i-1];

always @(posedge clk)
for (i = 0; i < B_REG; i = i + 1)
    b_reg[i] <= (i == 0) ? b : b_reg[i-1];

always @(posedge clk) begin
    m_reg <= a_reg[A_REG-1] * b_reg[B_REG-1];
    p_reg <= m_reg + pci;
end

assign a_passthrough = a_reg[A_REG-1];
assign p = p_reg;

endmodule

/*
 * 1 row of systolic Fir filetr, 1D
 */
module fir_row_5_cells
(
    input  wire          clk,
    input  wire signed [24:0] pixel_in,
    input  wire signed [17:0] coeff_0_i,

```

```

        input  wire signed [17:0]    coeff_1_i,
        input  wire signed [17:0]    coeff_2_i,
        input  wire signed [17:0]    coeff_3_i,
        input  wire signed [17:0]    coeff_4_i,
        input  wire signed [47:0]    pci_i,
        output wire signed [47:0]    p_o
    );

    localparam integer COEFF_REG_STAGES = 1;

    wire signed [24:0] pixel_0_to_1;
    wire signed [24:0] pixel_1_to_2;
    wire signed [24:0] pixel_2_to_3;
    wire signed [24:0] pixel_3_to_4;
    wire signed [24:0] pixel_4_unused;
    wire signed [47:0] p_0;
    wire signed [47:0] p_1;
    wire signed [47:0] p_2;
    wire signed [47:0] p_3;
    wire signed [47:0] p_4;

    dsp_25x18_passthrough #(
        .A_REG(1),
        .B_REG(COEFF_REG_STAGES)
    ) dsp_cell_0 (
        .clk(clk),
        .a(pixel_in),
        .b(coeff_0_i),
        .pci(pci_i),
        .a_passthrough(pixel_0_to_1),
        .p(p_0)
    );

    dsp_25x18_passthrough #(
        .A_REG(2),
        .B_REG(COEFF_REG_STAGES)
    ) dsp_cell_1 (
        .clk(clk),
        .a(pixel_0_to_1),
        .b(coeff_1_i),
        .pci(p_0),
        .a_passthrough(pixel_1_to_2),
        .p(p_1)
    );

    dsp_25x18_passthrough #(
        .A_REG(2),
        .B_REG(COEFF_REG_STAGES)
    ) dsp_cell_2 (
        .clk(clk),
        .a(pixel_1_to_2),
        .b(coeff_2_i),
        .pci(p_1),
        .a_passthrough(pixel_2_to_3),
        .p(p_2)
    );

    dsp_25x18_passthrough #(
        .A_REG(2),
        .B_REG(COEFF_REG_STAGES)
    ) dsp_cell_3 (

```

```

        .clk(clk),
        .a(pixel_2_to_3),
        .b(coeff_3_i),
        .pci(p_2),
        .a_passthrough(pixel_3_to_4),
        .p(p_3)
    );

dsp_25x18_passthrough #(
    .A_REG(2),
    .B_REG(COEFF_REG_STAGES)
) dsp_cell_4 (
    .clk(clk),
    .a(pixel_3_to_4),
    .b(coeff_4_i),
    .pci(p_3),
    .a_passthrough(pixel_4_unused),
    .p(p_4)
);

assign p_o = p_4;

endmodule

/*
 * full 5x5 window from the 5 rows
 */
module FIR_filter_5x5 #(
    parameter integer CTRL_DELAY = 12
) (
    input wire clk,
    input wire pixel_valid_i,
    input wire h_sync_i,
    input wire v_sync_i,
    input wire signed [24:0] pixel_0_i,
    input wire signed [24:0] pixel_1_i,
    input wire signed [24:0] pixel_2_i,
    input wire signed [24:0] pixel_3_i,
    input wire signed [24:0] pixel_4_i,
    input wire signed [15:0] coeff_00_i,
    input wire signed [15:0] coeff_01_i,
    input wire signed [15:0] coeff_02_i,
    input wire signed [15:0] coeff_03_i,
    input wire signed [15:0] coeff_04_i,
    input wire signed [15:0] coeff_10_i,
    input wire signed [15:0] coeff_11_i,
    input wire signed [15:0] coeff_12_i,
    input wire signed [15:0] coeff_13_i,
    input wire signed [15:0] coeff_14_i,
    input wire signed [15:0] coeff_20_i,
    input wire signed [15:0] coeff_21_i,
    input wire signed [15:0] coeff_22_i,
    input wire signed [15:0] coeff_23_i,
    input wire signed [15:0] coeff_24_i,
    input wire signed [15:0] coeff_30_i,
    input wire signed [15:0] coeff_31_i,
    input wire signed [15:0] coeff_32_i,
    input wire signed [15:0] coeff_33_i,
    input wire signed [15:0] coeff_34_i,
    input wire signed [15:0] coeff_40_i,
    input wire signed [15:0] coeff_41_i,

```

```

        input  wire signed [15:0]      coeff_42_i,
        input  wire signed [15:0]      coeff_43_i,
        input  wire signed [15:0]      coeff_44_i,
        output reg  [7:0]              fir_out_sat,
        output wire                    pixel_valid_out,
        output wire                    h_sync_out,
        output wire                    v_sync_out
    );

    wire signed [47:0] row0_p;
    wire signed [47:0] row1_p;
    wire signed [47:0] row2_p;
    wire signed [47:0] row3_p;
    wire signed [47:0] row4_p;
    wire signed [47:0] row_sum_total;
    wire signed [17:0] coeff_00_ext;
    wire signed [17:0] coeff_01_ext;
    wire signed [17:0] coeff_02_ext;
    wire signed [17:0] coeff_03_ext;
    wire signed [17:0] coeff_04_ext;
    wire signed [17:0] coeff_10_ext;
    wire signed [17:0] coeff_11_ext;
    wire signed [17:0] coeff_12_ext;
    wire signed [17:0] coeff_13_ext;
    wire signed [17:0] coeff_14_ext;
    wire signed [17:0] coeff_20_ext;
    wire signed [17:0] coeff_21_ext;
    wire signed [17:0] coeff_22_ext;
    wire signed [17:0] coeff_23_ext;
    wire signed [17:0] coeff_24_ext;
    wire signed [17:0] coeff_30_ext;
    wire signed [17:0] coeff_31_ext;
    wire signed [17:0] coeff_32_ext;
    wire signed [17:0] coeff_33_ext;
    wire signed [17:0] coeff_34_ext;
    wire signed [17:0] coeff_40_ext;
    wire signed [17:0] coeff_41_ext;
    wire signed [17:0] coeff_42_ext;
    wire signed [17:0] coeff_43_ext;
    wire signed [17:0] coeff_44_ext;

    /*
     * Coeffs are 16 bit bit dsp B reg is 18, extend for deterministic
     behaviour
     */
    assign coeff_00_ext = {{2{coeff_00_i[15]}}, coeff_00_i};
    assign coeff_01_ext = {{2{coeff_01_i[15]}}, coeff_01_i};
    assign coeff_02_ext = {{2{coeff_02_i[15]}}, coeff_02_i};
    assign coeff_03_ext = {{2{coeff_03_i[15]}}, coeff_03_i};
    assign coeff_04_ext = {{2{coeff_04_i[15]}}, coeff_04_i};
    assign coeff_10_ext = {{2{coeff_10_i[15]}}, coeff_10_i};
    assign coeff_11_ext = {{2{coeff_11_i[15]}}, coeff_11_i};
    assign coeff_12_ext = {{2{coeff_12_i[15]}}, coeff_12_i};
    assign coeff_13_ext = {{2{coeff_13_i[15]}}, coeff_13_i};
    assign coeff_14_ext = {{2{coeff_14_i[15]}}, coeff_14_i};
    assign coeff_20_ext = {{2{coeff_20_i[15]}}, coeff_20_i};
    assign coeff_21_ext = {{2{coeff_21_i[15]}}, coeff_21_i};
    assign coeff_22_ext = {{2{coeff_22_i[15]}}, coeff_22_i};
    assign coeff_23_ext = {{2{coeff_23_i[15]}}, coeff_23_i};
    assign coeff_24_ext = {{2{coeff_24_i[15]}}, coeff_24_i};

```

```

assign coeff_30_ext = {{2{coeff_30_i[15]}}}, coeff_30_i};
assign coeff_31_ext = {{2{coeff_31_i[15]}}}, coeff_31_i};
assign coeff_32_ext = {{2{coeff_32_i[15]}}}, coeff_32_i};
assign coeff_33_ext = {{2{coeff_33_i[15]}}}, coeff_33_i};
assign coeff_34_ext = {{2{coeff_34_i[15]}}}, coeff_34_i};
assign coeff_40_ext = {{2{coeff_40_i[15]}}}, coeff_40_i};
assign coeff_41_ext = {{2{coeff_41_i[15]}}}, coeff_41_i};
assign coeff_42_ext = {{2{coeff_42_i[15]}}}, coeff_42_i};
assign coeff_43_ext = {{2{coeff_43_i[15]}}}, coeff_43_i};
assign coeff_44_ext = {{2{coeff_44_i[15]}}}, coeff_44_i};

/*
 * Delay line for valid and sync
 */
generate
if (CTRL_DELAY > 0) begin : gen_ctrl_delay
    reg valid_delay [0:CTRL_DELAY-1];
    reg hsync_delay [0:CTRL_DELAY-1];
    reg vsync_delay [0:CTRL_DELAY-1];
    integer ctrl_idx;

    always @(posedge clk) begin
        for (ctrl_idx = 0; ctrl_idx < CTRL_DELAY; ctrl_idx = ctrl_idx
+ 1) begin
            valid_delay[ctrl_idx] <= (ctrl_idx == 0) ? pixel_valid_i :
valid_delay[ctrl_idx-1];
            hsync_delay[ctrl_idx] <= (ctrl_idx == 0) ? h_sync_i      :
hsync_delay[ctrl_idx-1];
            vsync_delay[ctrl_idx] <= (ctrl_idx == 0) ? v_sync_i      :
vsync_delay[ctrl_idx-1];
        end
    end

    assign pixel_valid_out = valid_delay[CTRL_DELAY-1];
    assign h_sync_out      = hsync_delay[CTRL_DELAY-1];
    assign v_sync_out      = vsync_delay[CTRL_DELAY-1];
end else begin : gen_ctrl_passthrough
    assign pixel_valid_out = pixel_valid_i;
    assign h_sync_out      = h_sync_i;
    assign v_sync_out      = v_sync_i;
end
endgenerate

fir_row_5_cells row_0 (
    .clk(clk),
    .pixel_in(pixel_0_i),
    .coeff_0_i(coeff_00_ext),
    .coeff_1_i(coeff_01_ext),
    .coeff_2_i(coeff_02_ext),
    .coeff_3_i(coeff_03_ext),
    .coeff_4_i(coeff_04_ext),
    .pci_i(48'sd0),
    .p_o(row0_p)
);

fir_row_5_cells row_1 (
    .clk(clk),
    .pixel_in(pixel_1_i),
    .coeff_0_i(coeff_10_ext),
    .coeff_1_i(coeff_11_ext),

```

```

        .coeff_2_i(coeff_12_ext),
        .coeff_3_i(coeff_13_ext),
        .coeff_4_i(coeff_14_ext),
        .pci_i(48'sd0),
        .p_o(row1_p)
    );

    fir_row_5_cells row_2 (
        .clk(clk),
        .pixel_in(pixel_2_i),
        .coeff_0_i(coeff_20_ext),
        .coeff_1_i(coeff_21_ext),
        .coeff_2_i(coeff_22_ext),
        .coeff_3_i(coeff_23_ext),
        .coeff_4_i(coeff_24_ext),
        .pci_i(48'sd0),
        .p_o(row2_p)
    );

    fir_row_5_cells row_3 (
        .clk(clk),
        .pixel_in(pixel_3_i),
        .coeff_0_i(coeff_30_ext),
        .coeff_1_i(coeff_31_ext),
        .coeff_2_i(coeff_32_ext),
        .coeff_3_i(coeff_33_ext),
        .coeff_4_i(coeff_34_ext),
        .pci_i(48'sd0),
        .p_o(row3_p)
    );

    fir_row_5_cells row_4 (
        .clk(clk),
        .pixel_in(pixel_4_i),
        .coeff_0_i(coeff_40_ext),
        .coeff_1_i(coeff_41_ext),
        .coeff_2_i(coeff_42_ext),
        .coeff_3_i(coeff_43_ext),
        .coeff_4_i(coeff_44_ext),
        .pci_i(48'sd0),
        .p_o(row4_p)
    );

    // First the five horizontal FIR rows are evaluated independently,
    // then their partial sums are accumulated into the final 5x5 result.
    assign row_sum_total = row0_p + row1_p + row2_p + row3_p + row4_p;

    // Drop the fractional Q8.8 part before clamping the result to 8-bit
    // range.
    always @(posedge clk) begin
        if (row_sum_total[47]) begin
            fir_out_sat <= 8'd0;
        end else if (~row_sum_total[46:16]) begin
            fir_out_sat <= 8'd255;
        end else begin
            fir_out_sat <= row_sum_total[15:8];
        end
    end

endmodule

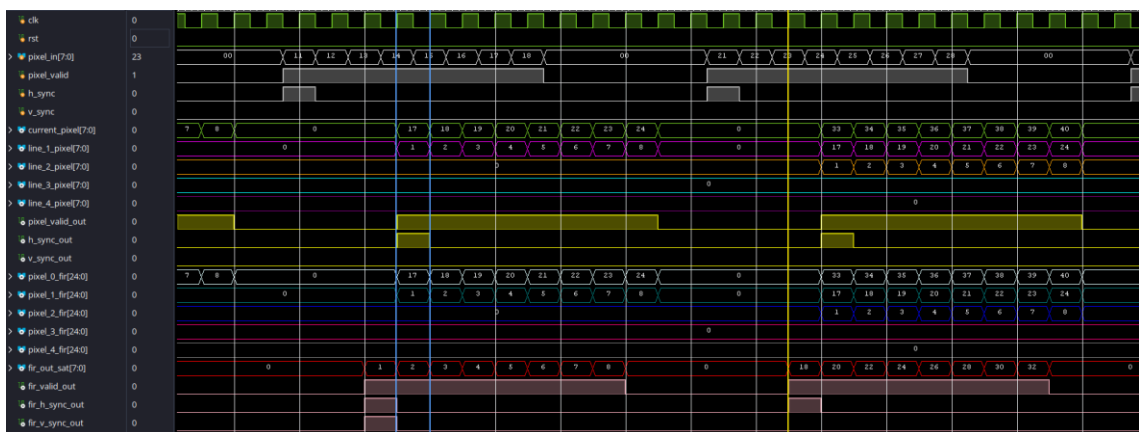
```

2.2.3.2. Szimulációs eredmények

Mivel a BRAM-ot előzőleg teszteltük és az RGB2Y modul is rendeltetés szerűen üzemelt, ezért a FIR szűrőt tesztelhettem már képi bemenettel is. Itt ugyan azt a Lena képet használtam, mint az RGB2Y modul esetén és 3 jólismert, szemmel is könnyen ellenőrizhető szűrők együtthatókészletét választottam. Ezek a Gauss, a Laplace szűrők és egy olyan együtthatókészlet, ami csak átenged. Ennek megfelelően az áteresztő szűrő szerepel az első képen, mellette jobbra a Gauss-szűrt (ennek tulajdonsága, hogy elmossa a képet), illetve utolsóként a Laplace szűrő (ez pedig egy éldetektorként funkcionál).



Amit különösen meg kell jegyezni az a FIR szűrő és a blank periódusok közötti összefüggés. Mivel ilyenkor 0 értékekkel kerülnek feltöltésre a BRAM pufferek, ezért az első 4 sor és első 4 oszlop minden esetben fekete pixelekké lesz feltöltve. Ez azonban egy 1600*900-as felbontás mellett elhanyagolható.



A FIR szűrőt önmagában a BRAM sorbufferekkel is teszteltem. Ebben a tesztben az 5x5 ablak együtthatóit úgy állítottam be, hogy a jobb legszélső oszlop elemei 1, a többi 0 legyen. Ezzel (és tudva azt, hogy az ablak a jobb alsó sarkos igazítással számol) ezt

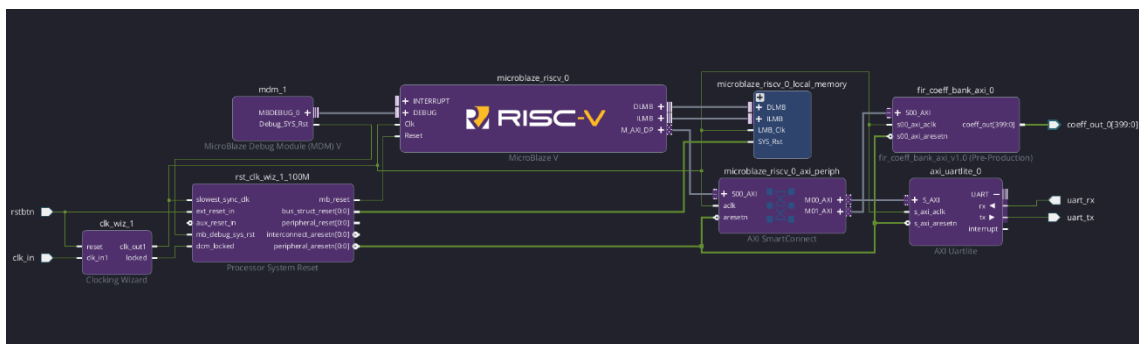
értjük el, hogy a FIR szűrő kimenetén a sorokban oszlopról oszlopra haladva az elemek összegét látjuk, például a jelölt $18 = 17+1$.

2.3. Processzoros alrendszer

A képfeldolgozó adatút mellett a rendszer egy külön processzoros alrendszert is tartalmaz, amelynek feladata a FIR szűrő együtthatóinak futásidejű kezelése. Ez az alrendszer egy MicroBlaze RISC-V processzorra épül, amely helyi memóriával, UART perifériával és egy egyedi AXI Lite csatolású együttható-regiszterbankkal kapcsolódik a rendszerhez. A processzor a soros porton keresztül fogadja a felhasználó által megadott szűrőegyütthatókat, majd ezeket az AXI buszon keresztül beírja a hardveres FIR szűrőhöz kapcsolódó regiszterekbe. Ennek köszönhetően a szűrőmaszk futás közben, újraszintetizálás nélkül is módosítható.

2.3.1. Blokkvázlat

A processzoros alrendszer központi eleme a MicroBlaze RISC-V processzor, amely egy helyi BRAM alapú memóriát használ a program és az adatok tárolására. A processzor AXI buszon keresztül két perifériát ér el: az egyik az AXI UARTLite, amely a soros kommunikációt biztosítja, a másik pedig a saját fejlesztésű `fir_coeff_bank_axi` periféria, amely 25 darab, egyenként 16 bites FIR együttható tárolását teszi lehetővé. A rendszer órajelét a processzoros rész számára egy 100 MHz-es órajel biztosítja. A blokkdiagramon jól látható, hogy a processzoros alrendszer nem közvetlenül a képfeldolgozó adatút része, hanem annak paraméterezését végzi. Ezen rendszerből szintetizált wrapper modul kerül példányosításra a top modulban, ahol a 400 bit széles (25×16 bit) `coeff_out` adatvonal szétbontódik és a megfelelő 16 széles csoportosítások a megfelelő bemeneteire kerülnek a FIR-szűrőnek.



2.3.2. Cím kiosztás

A processzor címtartományában a helyi memória és a két AXI periféria külön címet kapott. A helyi BRAM a 0x00000000 - 0x0000FFFF tartományban található, amelyet a processzor utasítás- és adatmemóriaként használ. Az AXI UARTLite periféria a 0x40600000 - 0x4060FFFF címtartományra van leképezve, ezen keresztül történik a soros kommunikáció a felhasználóval. A saját fejlesztésű fir_coeff_bank_axi periféria a 0x44A00000 - 0x44A0FFFF tartományban helyezkedik el. A szoftver a szűrőegységűtthetők ebbe a regiszterterbe írja be, így a hardveres FIR szűrő futás közben új paraméterekkel működtethető.

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0 (/microblaze_riscv_0/Data)					
/microblaze_riscv_0					
/microblaze_riscv_0/Data (32 address bits : 4G)					
/axi_uartlite_0/S_AXI	S_AXI	Reg	0x4060_0000	64K	0x4060_FFFF
/fir_coeff_bank_axi_0/S00_AXI	S00_AXI	S00_AXI_reg	0x44A0_0000	64K	0x44A0_FFFF
/microblaze_riscv_0_local_memory/dlmb_bram_if_cn	SLMB	Mem	0x0	64K	0xFFFF
Network 1 (/microblaze_riscv_0/Instruction)					
/microblaze_riscv_0					
/microblaze_riscv_0/Instruction (32 address bits : 4G)					
/microblaze_riscv_0_local_memory/ilmb_bram_if_cn	SLMB	Mem	0x0	64K	0xFFFF

2.3.3. Implementációs eredmények

Az implementációs eredmények alapján a terv a célhardveren sikeresen megvalósítható, és az időzítési követelményeket is teljesíti: a tervezői környezet nem jelzett sem failed route, sem timing hibát. A routed timing summary szerint a legrosszabb setup slack 1.986 ns, míg a legrosszabb hold slack 0.052 ns, tehát a rendszer időzítési szempontból kivitelezhető.

Az erőforrás-felhasználás alapján az implementált rendszer mérsékelt méretűnek tekinthető. Az implementáció után a terv összesen 3564 LUT-ot, 3875 flip-flopot, 19 BRAM blokkot és 29 DSP egységet használ. A DSP-felhasználásból az egészet az 5×5 FIR szűrő és a szürkeárnyaltos előfeldolgozás igényli, míg a BRAM blokkokat elsősorban a sorpufferek és a processzoros alrendszer helyi memóriája használja.

2.4. Teljes rendszer (top_module)

```
module hdmi_top(
    input wire      clk100M,
    input wire      rstbt,
    output wire [7:0] led_r,
    input wire [7:0] sw,
    inout wire [3:0] bt,
    input wire      uart_rxd,
    output wire     uart_txd,

    input wire      hdmi_rx_d0_p,
    input wire      hdmi_rx_d0_n,
    input wire      hdmi_rx_d1_p,
    input wire      hdmi_rx_d1_n,
    input wire      hdmi_rx_d2_p,
    input wire      hdmi_rx_d2_n,
    input wire      hdmi_rx_clk_p,
    input wire      hdmi_rx_clk_n,
    input wire      hdmi_rx_cec,
    output wire     hdmi_rx_hpd,
    input wire      hdmi_rx_scl,
    inout wire      hdmi_rx_sda,

    output wire     hdmi_tx_d0_p,
    output wire     hdmi_tx_d0_n,
    output wire     hdmi_tx_d1_p,
    output wire     hdmi_tx_d1_n,
    output wire     hdmi_tx_d2_p,
    output wire     hdmi_tx_d2_n,
    output wire     hdmi_tx_clk_p,
    output wire     hdmi_tx_clk_n,
    input wire      hdmi_tx_cec,
    input wire      hdmi_tx_hpdn,
    input wire      hdmi_tx_scl,
    input wire      hdmi_tx_sda
);

//*****
//* Generating the 200 MHz reference clock for the IDELAYCTRL.
//*****
//*****
wire clk200M;
wire pll_clkfb;
wire pll_locked;

PLLE2_BASE #(
    .BANDWIDTH("OPTIMIZED"),           // OPTIMIZED, HIGH, LOW
    .CLKFBOUT_MULT(10),                // Multiply value for all CLKOUT,
    (2-64)
    .CLKFBOUT_PHASE(0.0),              // Phase offset in degrees of
    CLKFB, (-360.000-360.000).
    .CLKIN1_PERIOD(1000.0 / 100.0),    // Input clock period in ns to ps
    resolution (i.e. 33.333 is 30 MHz).
    .CLKOUT0_DIVIDE(5),                // CLKOUT0_DIVIDE -
    CLKOUT5_DIVIDE: Divide amount for each CLKOUT (1-128)
);
```

```

        .CLKOUT1_DIVIDE(1),
        .CLKOUT2_DIVIDE(1),
        .CLKOUT3_DIVIDE(1),
        .CLKOUT4_DIVIDE(1),
        .CLKOUT5_DIVIDE(1),
        .CLKOUT0_DUTY_CYCLE(0.5),          // CLKOUT0_DUTY_CYCLE -
CLKOUT5_DUTY_CYCLE: Duty cycle for each CLKOUT (0.001-0.999).
        .CLKOUT1_DUTY_CYCLE(0.5),
        .CLKOUT2_DUTY_CYCLE(0.5),
        .CLKOUT3_DUTY_CYCLE(0.5),
        .CLKOUT4_DUTY_CYCLE(0.5),
        .CLKOUT5_DUTY_CYCLE(0.5),
        .CLKOUT0_PHASE(0.0),              // CLKOUT0_PHASE - CLKOUT5_PHASE:
Phase offset for each CLKOUT (-360.000-360.000).
        .CLKOUT1_PHASE(0.0),
        .CLKOUT2_PHASE(0.0),
        .CLKOUT3_PHASE(0.0),
        .CLKOUT4_PHASE(0.0),
        .CLKOUT5_PHASE(0.0),
        .DIVCLK_DIVIDE(1),                // Master division value, (1-56)
        .REF_JITTER1(0.0),                // Reference input jitter in UI,
(0.000-0.999).
        .STARTUP_WAIT("FALSE")           // Delay DONE until PLL Locks,
("TRUE"/"FALSE")
    ) clk_generator1 (
        .CLKOUT0(clk200M),                // 1-bit output: CLKOUT0
        .CLKOUT1(),                       // 1-bit output: CLKOUT1
        .CLKOUT2(),                       // 1-bit output: CLKOUT2
        .CLKOUT3(),                       // 1-bit output: CLKOUT3
        .CLKOUT4(),                       // 1-bit output: CLKOUT4
        .CLKOUT5(),                       // 1-bit output: CLKOUT5
        .CLKFBOUT(pll_clkfb),             // 1-bit output: Feedback clock
        .LOCKED(pll_locked),              // 1-bit output: LOCK
        .CLKIN1(clk100M),                 // 1-bit input: Input clock
        .PWRDWN(1'b0),                   // 1-bit input: Power-down
        .RST(rstbt),                      // 1-bit input: Reset
        .CLKFBIN(pll_clkfb)               // 1-bit input: Feedback clock
    );

wire rst;
assign rst = ~pll_locked;

wire clk_200M;
BUFG BUFG_200M (
    .O(clk_200M),
    .I(clk200M)
);

wire rx_clk, rx_clk_5x;
wire [7:0] rx_red, rx_green, rx_blue;
wire rx_dv, rx_hs, rx_vs;
wire [5:0] rx_status;
hdmi_rx hdmi_rx_0 (
    .clk_200M(clk_200M),
    .rst(rst),
    .hdmi_rx_cec(hdmi_rx_cec),
    .hdmi_rx_hpd(hdmi_rx_hpd),
    .hdmi_rx_scl(hdmi_rx_scl),
    .hdmi_rx_sda(hdmi_rx_sda),

```

```

        .hdmi_rx_clk_p(hdmi_rx_clk_p),
        .hdmi_rx_clk_n(hdmi_rx_clk_n),
        .hdmi_rx_d0_p(hdmi_rx_d0_p),
        .hdmi_rx_d0_n(hdmi_rx_d0_n),
        .hdmi_rx_d1_p(hdmi_rx_d1_p),
        .hdmi_rx_d1_n(hdmi_rx_d1_n),
        .hdmi_rx_d2_p(hdmi_rx_d2_p),
        .hdmi_rx_d2_n(hdmi_rx_d2_n),
        .rx_clk(rx_clk),
        .rx_clk_5x(rx_clk_5x),
        .rx_red(rx_red),
        .rx_green(rx_green),
        .rx_blue(rx_blue),
        .rx_dv(rx_dv),
        .rx_hs(rx_hs),
        .rx_vs(rx_vs),
        .rx_status(rx_status)
    );

    localparam integer FIR_LINE_WIDTH = 2048;
    localparam integer FIR_ADDR_WIDTH = 11;

    wire rgb2y_dv;
    wire rgb2y_hs;
    wire rgb2y_vs;
    wire [7:0] rgb2y_y;
    wire [399:0] coeff_wire;
    wire signed [15:0] fir_coeff_00;
    wire signed [15:0] fir_coeff_01;
    wire signed [15:0] fir_coeff_02;
    wire signed [15:0] fir_coeff_03;
    wire signed [15:0] fir_coeff_04;
    wire signed [15:0] fir_coeff_10;
    wire signed [15:0] fir_coeff_11;
    wire signed [15:0] fir_coeff_12;
    wire signed [15:0] fir_coeff_13;
    wire signed [15:0] fir_coeff_14;
    wire signed [15:0] fir_coeff_20;
    wire signed [15:0] fir_coeff_21;
    wire signed [15:0] fir_coeff_22;
    wire signed [15:0] fir_coeff_23;
    wire signed [15:0] fir_coeff_24;
    wire signed [15:0] fir_coeff_30;
    wire signed [15:0] fir_coeff_31;
    wire signed [15:0] fir_coeff_32;
    wire signed [15:0] fir_coeff_33;
    wire signed [15:0] fir_coeff_34;
    wire signed [15:0] fir_coeff_40;
    wire signed [15:0] fir_coeff_41;
    wire signed [15:0] fir_coeff_42;
    wire signed [15:0] fir_coeff_43;
    wire signed [15:0] fir_coeff_44;

    wire [7:0] current_pixel;
    wire [7:0] line_1_pixel;
    wire [7:0] line_2_pixel;
    wire [7:0] line_3_pixel;
    wire [7:0] line_4_pixel;
    wire linebuf_dv;
    wire linebuf_hs;
    wire linebuf_vs;

```

```

wire signed [24:0] fir_pixel_0;
wire signed [24:0] fir_pixel_1;
wire signed [24:0] fir_pixel_2;
wire signed [24:0] fir_pixel_3;
wire signed [24:0] fir_pixel_4;

wire [7:0] fir_out_sat;
wire fir_dv;
wire fir_hs;
wire fir_vs;

wire [7:0] tx_red;
wire [7:0] tx_green;
wire [7:0] tx_blue;
wire tx_dv;
wire tx_hs;
wire tx_vs;

cpu_hdmi_system_sv processor_inst (
    .rstbtn(rstbt),
    .clk_in(clk100M),
    .uart_rx(uart_rxd),
    .uart_tx(uart_txd),
    .coeff_out_0(coeff_wire)
);

assign fir_coeff_00 = coeff_wire[15:0];
assign fir_coeff_01 = coeff_wire[31:16];
assign fir_coeff_02 = coeff_wire[47:32];
assign fir_coeff_03 = coeff_wire[63:48];
assign fir_coeff_04 = coeff_wire[79:64];
assign fir_coeff_10 = coeff_wire[95:80];
assign fir_coeff_11 = coeff_wire[111:96];
assign fir_coeff_12 = coeff_wire[127:112];
assign fir_coeff_13 = coeff_wire[143:128];
assign fir_coeff_14 = coeff_wire[159:144];
assign fir_coeff_20 = coeff_wire[175:160];
assign fir_coeff_21 = coeff_wire[191:176];
assign fir_coeff_22 = coeff_wire[207:192];
assign fir_coeff_23 = coeff_wire[223:208];
assign fir_coeff_24 = coeff_wire[239:224];
assign fir_coeff_30 = coeff_wire[255:240];
assign fir_coeff_31 = coeff_wire[271:256];
assign fir_coeff_32 = coeff_wire[287:272];
assign fir_coeff_33 = coeff_wire[303:288];
assign fir_coeff_34 = coeff_wire[319:304];
assign fir_coeff_40 = coeff_wire[335:320];
assign fir_coeff_41 = coeff_wire[351:336];
assign fir_coeff_42 = coeff_wire[367:352];
assign fir_coeff_43 = coeff_wire[383:368];
assign fir_coeff_44 = coeff_wire[399:384];

assign fir_pixel_0 = {17'd0, current_pixel};
assign fir_pixel_1 = {17'd0, line_1_pixel};
assign fir_pixel_2 = {17'd0, line_2_pixel};
assign fir_pixel_3 = {17'd0, line_3_pixel};
assign fir_pixel_4 = {17'd0, line_4_pixel};

rgb2y rgb2y_0(
    .clk(rx_clk),

```

```

        .kr_i(18'sd27865),
        .kb_i(18'sd9463),
        .dv_i(rx_dv),
        .hs_i(rx_hs),
        .vs_i(rx_vs),
        .r_i(rx_red),
        .g_i(rx_green),
        .b_i(rx_blue),
        .dv_o(rgb2y_dv),
        .hs_o(rgb2y_hs),
        .vs_o(rgb2y_vs),
        .y_o(rgb2y_y)
    );

    bram_line_buffer #(
        .DATA_WIDTH(8),
        .LINE_WIDTH(FIR_LINE_WIDTH),
        .ADDR_WIDTH(FIR_ADDR_WIDTH)
    ) line_buffer_0 (
        .clk(rx_clk),
        .rst(rst),
        .pixel_in(rgb2y_y),
        .pixel_valid(rgb2y_dv),
        .h_sync(rgb2y_hs),
        .v_sync(rgb2y_vs),
        .current_pixel(current_pixel),
        .line_1_pixel(line_1_pixel),
        .line_2_pixel(line_2_pixel),
        .line_3_pixel(line_3_pixel),
        .line_4_pixel(line_4_pixel),
        .pixel_valid_out(linebuf_dv),
        .h_sync_out(linebuf_hs),
        .v_sync_out(linebuf_vs)
    );

    FIR_filter_5x5 #(
        .CTRL_DELAY(8)
    ) fir_0 (
        .clk(rx_clk),
        .pixel_valid_i(linebuf_dv),
        .h_sync_i(linebuf_hs),
        .v_sync_i(linebuf_vs),
        .pixel_0_i(fir_pixel_0),
        .pixel_1_i(fir_pixel_1),
        .pixel_2_i(fir_pixel_2),
        .pixel_3_i(fir_pixel_3),
        .pixel_4_i(fir_pixel_4),
        .coeff_00_i(fir_coeff_00),
        .coeff_01_i(fir_coeff_01),
        .coeff_02_i(fir_coeff_02),
        .coeff_03_i(fir_coeff_03),
        .coeff_04_i(fir_coeff_04),
        .coeff_10_i(fir_coeff_10),
        .coeff_11_i(fir_coeff_11),
        .coeff_12_i(fir_coeff_12),
        .coeff_13_i(fir_coeff_13),
        .coeff_14_i(fir_coeff_14),
        .coeff_20_i(fir_coeff_20),
        .coeff_21_i(fir_coeff_21),
        .coeff_22_i(fir_coeff_22),
        .coeff_23_i(fir_coeff_23),

```

```

        .coeff_24_i(fir_coeff_24),
        .coeff_30_i(fir_coeff_30),
        .coeff_31_i(fir_coeff_31),
        .coeff_32_i(fir_coeff_32),
        .coeff_33_i(fir_coeff_33),
        .coeff_34_i(fir_coeff_34),
        .coeff_40_i(fir_coeff_40),
        .coeff_41_i(fir_coeff_41),
        .coeff_42_i(fir_coeff_42),
        .coeff_43_i(fir_coeff_43),
        .coeff_44_i(fir_coeff_44),
        .fir_out_sat(fir_out_sat),
        .pixel_valid_out(fir_dv),
        .h_sync_out(fir_hs),
        .v_sync_out(fir_vs)
    );

    assign tx_dv      = fir_dv;
    assign tx_hs      = fir_hs;
    assign tx_vs      = fir_vs;
    assign tx_red     = fir_out_sat;
    assign tx_green   = fir_out_sat;
    assign tx_blue    = fir_out_sat;

    hdmi_tx hdmi_tx_0(
        .tx_clk(rx_clk),
        .tx_clk_5x(rx_clk_5x),
        .rst(rst),
        .tx_red(tx_red),
        .tx_green(tx_green),
        .tx_blue(tx_blue),
        .tx_dv(tx_dv),
        .tx_hs(tx_hs),
        .tx_vs(tx_vs),
        .hdmi_tx_clk_p(hdmi_tx_clk_p),
        .hdmi_tx_clk_n(hdmi_tx_clk_n),
        .hdmi_tx_d0_p(hdmi_tx_d0_p),
        .hdmi_tx_d0_n(hdmi_tx_d0_n),
        .hdmi_tx_d1_p(hdmi_tx_d1_p),
        .hdmi_tx_d1_n(hdmi_tx_d1_n),
        .hdmi_tx_d2_p(hdmi_tx_d2_p),
        .hdmi_tx_d2_n(hdmi_tx_d2_n)
    );

    assign led_r = {pll_locked, 1'b0, rx_status};

endmodule

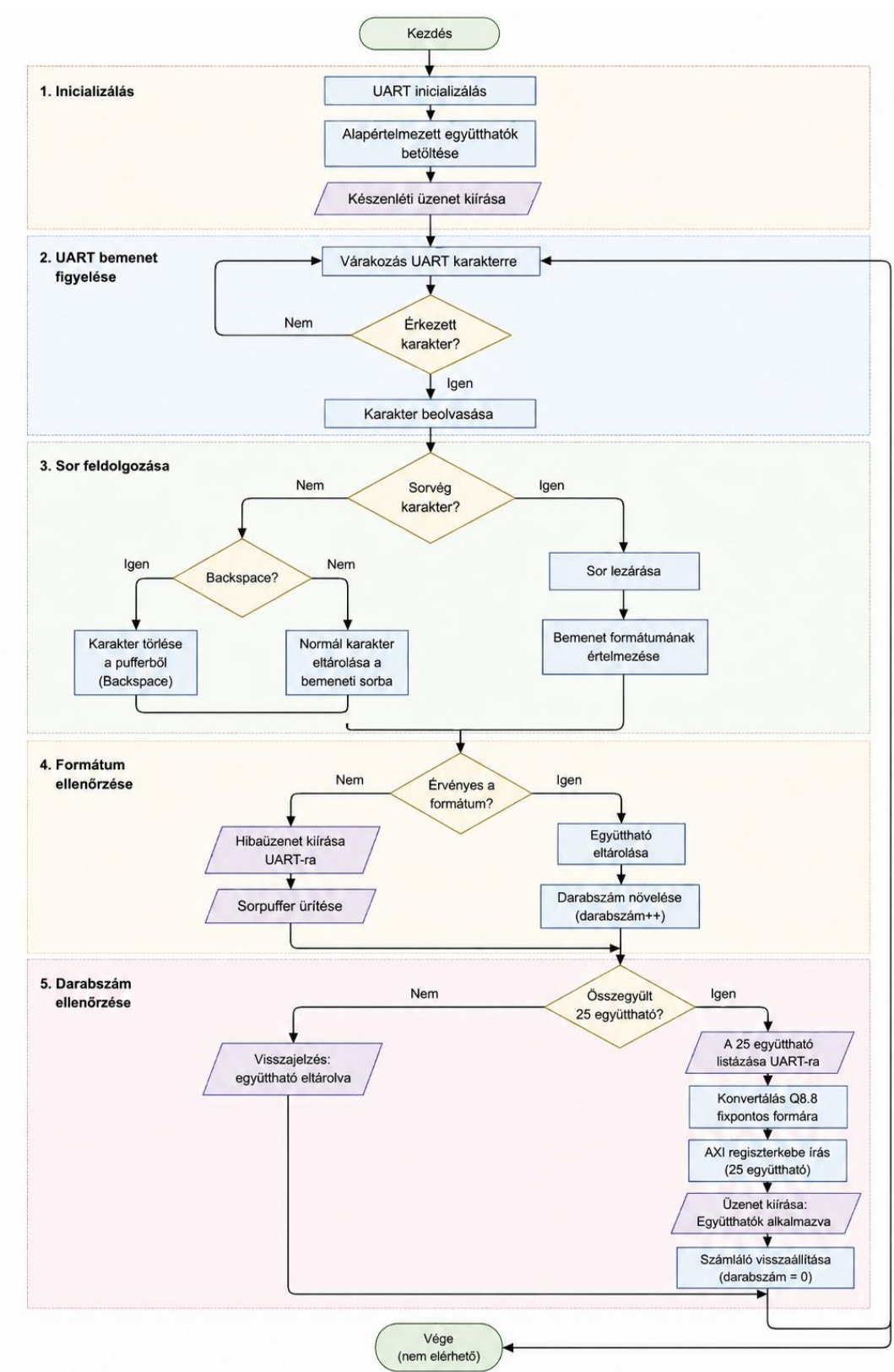
```


3. Szoftver leírás

3.1. A MicroBlaze szoftvere

A MicroBlaze szoftvere egy folyamatosan futó, soros kommunikációra épülő vezérlőprogram. Induláskor inicializálja az UART perifériát, majd betölti az alapértelmezett FIR együtthatókat a hardveres regiszterbankba. Ez az alapállapot egy áteresztő szűrőnek felel meg, ahol minden együttható nulla, kivéve a jobb alsó sarokhoz tartozó elemet, amely +1.0000 értéket kap. Ezt követően a program a soros vonalon érkező karaktereket figyeli, soronként értelmezi a felhasználói bemenetet, majd 25 darab együttható összegyűjtése után ezeket fixpontos formára alakítja és beírja az együttható bank AXI regisztereibe.

3.1.1. Folyamatábra



3.1.2. Megvalósított függvények

decimal4_to_q8_8_raw(): A felhasználó által megadott, négy tizedesjegyes decimális értéket Q8.8 fixpontos formára alakítja. Erre azért van szükség, mert a hardveres FIR szűrő együtthatói egész számként kerülnek beírásra a regiszterekbe, miközben törtreszes értékeket is kezelni kell.

load_default_coefficients(): Inicializálja a FIR együtthatókat. A függvény minden regisztert nullára állít, majd a jobb alsó sarokhoz tartozó együtthatót 1.0 értékre állítja, így a rendszer induláskor áteresztő szűrőként működik.

main(): A program fő vezérlőfüggvénye. Elvégzi az UART inicializálását, betölti az alapértelmezett együtthatókat, majd végtelen ciklusban fogadja és feldolgozza a felhasználó által beírt karaktereket. A függvény ellenőrzi a bemeneti formátumot, eltárolja a 25 darab együtthatót, majd azokat fixpontos formára alakítva beírja az AXI regiszterekbe.

A kódban egyébként megtalálható függvények, mint például az UART vezérlése vagy az együttható bankba való írás mind a Xilinx által generált Board Support Package részét képezik.

3.1.3. Kód

```
#include "xparameters.h"
#include "xil_io.h"
#include "xil_printf.h"
#include "xuartlite_1.h"

static int decimal4_to_q8_8_raw(int decimal4_value)
{
    long long scaled;

    /* Round the 4-decimal fixed-point input to the nearest Q8.8
integer. */
    if (decimal4_value >= 0) {
        scaled = (long long)decimal4_value * 256 + 5000;
    } else {
        scaled = (long long)decimal4_value * 256 - 5000;
    }

    return (int)(scaled / 10000);
}

static void load_default_coefficients(void)
```

```

{
    int coeff_index;

    /* Clear all 25 coefficient registers before loading the identity
tap. */
    for (coeff_index = 0; coeff_index < 25; ++coeff_index) {
        Xil_Out32(XPAR_FIR_COEFF_BANK_AXI_0_BASEADDR + 4 *
coeff_index, 0);
    }

    /*
    * The streaming FIR uses the bottom-right tap as the "current"
sample
    * location, so an identity kernel is EH 24 = +1.0000.
    */
    Xil_Out32(XPAR_FIR_COEFF_BANK_AXI_0_BASEADDR + 4 * 24, 256);
}

int main(void)
{
    u8 received_byte;
    char input_line[64];
    int input_length = 0;
    int decimal4_coefficients[25];
    int stored_coefficients = 0;
    int coeff_index;

    XUartLite_SetControlReg(XPAR_XUARTLITE_0_BASEADDR,
XUL_CR_FIFO_RX_RESET |
XUL_CR_FIFO_TX_RESET);

    load_default_coefficients();

    xil_printf("\r\nUART FIR coefficient loader ready.\r\n");
    xil_printf("Send 25 coefficients line by line.\r\n");
    xil_printf("Accepted input: +0.0011 or EH 0: +0.0011\r\n");
    xil_printf("Default kernel after reset: EH 24 = +1.0000, others
0.\r\n\r\n");

    while (1) {
        /* Poll the UART until a new character arrives. */
        while (XUartLite_IsReceiveEmpty(XPAR_XUARTLITE_0_BASEADDR)) {
        }

        received_byte = XUartLite_RecvByte(XPAR_XUARTLITE_0_BASEADDR);

        if (received_byte == '\r' || received_byte == '\n') {
            int sign = 1;
            int int_part = 0;
            int frac_part = 0;
            int frac_digits = 0;
            int value = 0;
            int valid = 1;
            int found_number = 0;
            char *cursor;

            while
(XUartLite_IsTransmitFull(XPAR_XUARTLITE_0_BASEADDR)) {
            }
            XUartLite_SendByte(XPAR_XUARTLITE_0_BASEADDR, '\r');

```

```

        while
(XUartLite_IsTransmitFull(XPAR_XUARTLITE_0_BASEADDR)) {
    }
    XUartLite_SendByte(XPAR_XUARTLITE_0_BASEADDR, '\n');

    input_line[input_length] = '\0';
    cursor = input_line;

    /* Accept optional leading whitespace and the "EH n:"
prefix. */
    while (*cursor == ' ' || *cursor == '\t') {
        ++cursor;
    }

    if ((cursor[0] == 'E' || cursor[0] == 'e') &&
        (cursor[1] == 'H' || cursor[1] == 'h')) {
        cursor += 2;

        while (*cursor == ' ' || *cursor == '\t') {
            ++cursor;
        }

        while (*cursor >= '0' && *cursor <= '9') {
            ++cursor;
        }

        while (*cursor == ' ' || *cursor == '\t') {
            ++cursor;
        }

        if (*cursor == ':') {
            ++cursor;
        }
    }

    while (*cursor == ' ' || *cursor == '\t') {
        ++cursor;
    }

    if (*cursor == '+') {
        ++cursor;
    } else if (*cursor == '-') {
        sign = -1;
        ++cursor;
    }

    while (*cursor >= '0' && *cursor <= '9') {
        found_number = 1;
        int_part = int_part * 10 + (*cursor - '0');
        ++cursor;
    }

    if (*cursor == '.') {
        ++cursor;

        while (*cursor >= '0' && *cursor <= '9') {
            if (frac_digits < 4) {
                frac_part = frac_part * 10 + (*cursor - '0');
            }
            ++frac_digits;
            ++cursor;
        }
    }
}

```

```

    }
}

while (frac_digits < 4) {
    frac_part *= 10;
    ++frac_digits;
}

while (*cursor == ' ' || *cursor == '\t') {
    ++cursor;
}

if (*cursor != '\0') {
    valid = 0;
}

if (!found_number) {
    valid = 0;
}

/* More than 4 fractional digits would not match the
decimal4 format. */
if (frac_digits > 4) {
    valid = 0;
}

if (valid) {
    /* Store coefficients as signed integers scaled by
10^4. */
    value = sign * (int_part * 10000 + frac_part);
    decimal4_coefficients[stored_coefficients] = value;
    ++stored_coefficients;

    xil_printf("Stored coefficient %d/25\r\n",
stored_coefficients);

    if (stored_coefficients == 25) {
        xil_printf("\r\nFull coefficient set
received.\r\n");

        for (coeff_index = 0; coeff_index < 25;
++coeff_index) {
            int abs_value =
decimal4_coefficients[coeff_index];
            char sign_char = '+';

            if (abs_value < 0) {
                sign_char = '-';
                abs_value = -abs_value;
            }

            xil_printf("EH %d: %c%d.%04d\r\n",
coeff_index,
sign_char,
abs_value / 10000,
abs_value % 10000);
        }

        /* Convert and write the full 5x5 coefficient set
in register order. */

```

```

        for (coeff_index = 0; coeff_index < 25;
++coeff_index) {
            int q8_8_raw = decimal4_to_q8_8_raw(
                decimal4_coefficients[coeff_index]);
            Xil_Out32(XPAR_FIR_COEFF_BANK_AXI_0_BASEADDR +
4 * coeff_index,
                (u32)q8_8_raw);
        }

        xil_printf("\r\nCoefficients applied to
FIR.\r\n");
        xil_printf("Ready for next 25 coefficients.\r\n");
        stored_coefficients = 0;
    }
    } else if (input_length != 0) {
        xil_printf("Invalid format. Use +0.0011 or EH n:
+0.0011\r\n");
    }

    /* Start collecting the next input line after Enter. */
    input_length = 0;
    } else if ((received_byte == 8 || received_byte == 127) &&
input_length > 0) {
        --input_length;

        /* Echo backspace in a terminal-friendly way. */
        while
(XUartLite_IsTransmitFull(XPAR_XUARTLITE_0_BASEADDR)) {
        }
        XUartLite_SendByte(XPAR_XUARTLITE_0_BASEADDR, 8);
        while
(XUartLite_IsTransmitFull(XPAR_XUARTLITE_0_BASEADDR)) {
        }
        XUartLite_SendByte(XPAR_XUARTLITE_0_BASEADDR, ' ');
        while
(XUartLite_IsTransmitFull(XPAR_XUARTLITE_0_BASEADDR)) {
        }
        XUartLite_SendByte(XPAR_XUARTLITE_0_BASEADDR, 8);
    } else if (input_length < (int)sizeof(input_line) - 1) {
        input_line[input_length] = (char)received_byte;
        ++input_length;

        /* Echo regular characters so the typed coefficient is
visible on UART. */
        while
(XUartLite_IsTransmitFull(XPAR_XUARTLITE_0_BASEADDR)) {
        }
        XUartLite_SendByte(XPAR_XUARTLITE_0_BASEADDR,
received_byte);
    } else {
        xil_printf("\r\nLine too long.\r\n");
        input_length = 0;
    }
}

return 0;
}

```

3.1.4. Tesztelés

Bemenet	Elvárt működés	Megfigyelés	Eredmény
RST gomb megnyomása	Áteresztő szűrő betöltése	Kiterjesztett képernyő megjelenik változatlanul	PASSED
Laplace szűrő együtthatóinak megadása helyesen	Éldetektor a képre	FIR szűrőnél megfigyelt Laplace szűrt kép, csak folytonos stream mellett	PASSED
Bemeneti formátum: abc	Hibaüzenet, nem fogadja el	Invalid format	PASSED
Törlés gépelsé közben	Beírt karakter törlése	Beírt karakter törlődött	PASSED
Kevesebb mint 25 együttható megadása (24)	Végtelen ideig vár az utolsóra	Képben nem történik változás	PASSED
Nincs megadva előjel	Pozitívnak kell vegye	Pozitívnak veszi, ha kiolvassuk a regisztert	PASSED
Egész rész le hagyása .1234	Hibaüzenet	Invalid format	PASSED